SIMATS ENGINEERING

ASSESSMENT TOOL 2 — ANNEXURE A

COURSE CODE: CSA14 COURSE NAME: Compiler Design

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

INDUSTRY PROBLEM-SOLVING TASK — SOLUTIONS

Register Number: 192511124

Q1. Smart Retail Billing System (10 Marks)

Concepts Covered: Intermediate Languages, Declarations, Assignment Statements

Source statements:

```
total_cost = price * quantity
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

(i) Declarations

```
float price;          // unit price of the product
int    quantity;      // number of units purchased
float  discount;      // discount rate (0.0 - 1.0)
float  total_cost;    // price * quantity
float  discount_value;// discount amount deducted
float  final_bill;    // amount payable by customer
```

(ii) Three-Address Code (TAC)

```
t1 = price * quantity
total_cost = t1
t2 = total_cost * discount
discount_value = t2
t3 = total_cost - discount_value
final_bill = t3
```

(iii) Quadruples

#	Op	Arg1	Arg2	Result
0	*	price	quantity	t1
1	=	t1	-	total_cost
2	*	total_cost	discount	t2
3	=	t2	-	discount_value

#	Op	Arg1	Arg2	Result
4	-	total_cost	discount_value	t3
5	=	t3	-	final_bill

Triples (result is implicit — referred to by statement number):

#	Op	Arg1	Arg2
(0)	*	price	quantity
(1)	=total_cost	(0)	-
(2)	*	(1)	discount
(3)	=discount_value	(2)	-
(4)	-	(1)	(3)
(5)	=final_bill	(4)	-

Indirect Triples (a statement pointer list references the triple table, allowing reordering without rewriting triples):

Statement Pointer	→ Triple #
(100)	(0)
(101)	(1)
(102)	(2)
(103)	(3)
(104)	(4)
(105)	(5)

(iv) Optimization Opportunities

- Copy propagation: t1, t2, t3 are simple copies immediately assigned to total_cost, discount_value and final_bill — the temporaries are redundant and can be eliminated.
- total_cost is a common sub-expression reused in statement 2 (discount_value) and statement 3 (final_bill) — it is already computed once, so no recomputation is needed if it is retained in a register.
- No dead code or loop-invariant computation exists here, since every value is used downstream.

(v) Optimized Intermediate Code

```
total_cost = price * quantity
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

Justification: eliminating the temporaries t1, t2 and t3 via copy propagation reduces the code from 6 three-address instructions to 3, removing 3 redundant copy operations and 3 unnecessary temporary variables while keeping total_cost available in a register for reuse (common sub-expression). This lowers both instruction count and register/memory pressure without changing program semantics.

Q2. University Scholarship Eligibility System (10 Marks)

Concepts Covered: Boolean Expressions, Backpatching

(i) Boolean Expression

```
eligible = (CGPA > 8.5) AND ( (Family_Income < 300000) OR (Sports_Quota == TRUE) )
```

(ii) Intermediate Code using Short-Circuit Evaluation

Using the classical translation scheme with truelist (truelist) and falselist (falselist) attributes, code is generated so that each Boolean sub-expression jumps directly to the true or false target instead of computing a 0/1 value:

```
100: if CGPA > 8.5 goto 102
101: goto Lfalse            // CGPA test fails -> short circuit AND
102: if Family_Income < 300000 goto Ltrue   // OR: first disjunct true -> short
circuit
103: if Sports_Quota == TRUE goto Ltrue     // OR: second disjunct
104: goto Lfalse            // both OR operands false
```

(iii) Backpatching to Resolve Forward Jumps

At code-generation time the targets Ltrue and Lfalse are not yet known, so the jump instructions at 101, 102, 103 and 104 are generated with an empty target and their instruction numbers are placed on falselist / truelist chains:

- E1.truelist = {100} (jump to 102 already resolved to next instruction)
- E1.falselist = {101}
- E2(OR).truelist = {102, 103}
- E2(OR).falselist = {104}
- E(AND).truelist = E2.truelist = {102, 103}
- E(AND).falselist = merge(E1.falselist, E2.falselist) = {101, 104}

When the statement that follows is generated, backpatch(E.truelist, Ltrue) and backpatch(E.falselist, Lfalse) fill in the actual target addresses (105 and 107 respectively) into every instruction on the respective lists.

(iv) Final Labelled Intermediate Code

```
100: if CGPA > 8.5 goto 102
101: goto 107
102: if Family_Income < 300000 goto 105
103: if Sports_Quota == TRUE goto 105
104: goto 107
105: eligible = TRUE
106: goto 108
107: eligible = FALSE
108: // eligible now holds final result
```

Here Ltrue resolves to 105 and Lfalse resolves to 107; backpatch(E.truelist,105) fills instructions 102 and 103, and backpatch(E.falselist,107) fills instructions 101 and 104.

(v) Control-Flow Graph

Basic blocks and control edges:

Block	Instructions	True Edge →	False Edge →
B1	100: if CGPA>8.5	B2 (102)	B5 (101→107)
B2	102: if Income<300000	B4 (105)	B3 (103)
B3	103: if SportsQuota==TRUE	B4 (105)	B5 (104→107)
B4	105: eligible=TRUE; 106: goto 108	B6 (108)	-
B5	107: eligible = FALSE	B6 (108)	-
B6	108: (join / next statement)	-	-

Flow: B1 → (true) B2, B1 → (false) B5; B2 → (true) B4, B2 → (false) B3; B3 → (true) B4, B3 → (false) B5; B4 → B6; B5 → B6.

Q3. Smart Parking Management System (10 Marks)

Concepts Covered: Case Statements, Intermediate Languages

```
switch(vehicle_type) {
  case 1: TWO_WHEELER;
  case 2: CAR;
  case 3: BUS;
  case 4: EMERGENCY_VEHICLE;
}
```

(i) Three-Address Code using Conditional Jumps

```
100: if vehicle_type == 1 goto 105
101: if vehicle_type == 2 goto 107
102: if vehicle_type == 3 goto 109
103: if vehicle_type == 4 goto 111
104: goto 113                      // default / no match
105: TWO_WHEELER
106: goto 113
107: CAR
108: goto 113
109: BUS
110: goto 113
111: EMERGENCY_VEHICLE
112: goto 113
113: // next statement
```

(ii) Equivalent Jump-Table Implementation

```
// Assumes vehicle_type in range 1..4, contiguous
t1 = vehicle_type - 1
```

```
if (t1 < 0) goto L_default
if (t1 > 3) goto L_default
t2 = t1 * 4              // width of an address entry
t3 = JumpTable[t2]       // fetch target address
goto *t3                 // indirect jump

JumpTable:  [ L_TWO_WHEELER, L_CAR, L_BUS, L_EMERGENCY ]

L_TWO_WHEELER: TWO_WHEELER;  goto L_END
L_CAR:         CAR;          goto L_END
L_BUS:         BUS;          goto L_END
L_EMERGENCY:   EMERGENCY_VEHICLE; goto L_END
L_default:     // invalid vehicle_type
L_END:
```

(iii) Quadruple Representation (conditional-jump form)

#	Op	Arg1	Arg2	Result
100	==	vehicle_type	1	goto 105
101	==	vehicle_type	2	goto 107
102	==	vehicle_type	3	goto 109
103	==	vehicle_type	4	goto 111
104	goto	-	-	113
105	call	TWO_WHEELER	-	-
107	call	CAR	-	-
109	call	BUS	-	-
111	call	EMERGENCY_VEHICLE	-	-

(iv) Conditional Branching vs Jump-Table

Aspect	Conditional Branching	Jump Table
Time complexity	O(n) – up to n comparisons	O(1) – single indexed jump
Code size	Grows linearly with number of cases	Compact; one table + one jump
Best suited for	Few / sparse / non-contiguous case values	Many, dense, contiguous case values
Branch prediction	Can suffer mispredictions with many cases	Predictable single indirect jump
Memory overhead	Low (no table)	Extra memory for the address table

(v) Recommendation

For a large-scale parking system with many vehicle categories (two-wheeler, car, bus, emergency vehicle, and future additions such as trucks, EVs, etc.), a jump-table implementation is recommended. Since vehicle_type values are small dense integers, the jump table gives constant-time O(1) dispatch regardless of the number of

categories, keeping response time predictable as the system scales, whereas a chain of conditional branches would grow linearly slower and harder to maintain.

Q4. Online Food Delivery Processing System (10 Marks)

Concepts Covered: Procedure Calls, Activation Records, Intermediate Code

```
amount = calculateBill(order)
applyCoupon(amount)
generateInvoice(amount)
```

(i) Intermediate Code for Procedure Calls

```
param order
t1 = call calculateBill, 1
amount = t1
param amount
call applyCoupon, 1
param amount
call generateInvoice, 1
```

(ii) Parameter Passing and Return-Value Handling

- Each actual parameter is pushed/queued with a param instruction immediately before the call; the callee reads them from its activation record.
- calculateBill(order) returns a value: the call instruction's target temporary (t1) receives it, and t1 is then copied into amount.
- applyCoupon(amount) and generateInvoice(amount) are void procedures — they are called for effect and no return value is captured.

(iii) Activation Record (Stack Frame) per Call

Field	calculateBill(order)	applyCoupon(amount)	generateInvoice(amount)
Return value slot	t1 (bill amount)	– (void)	– (void)
Actual parameters	order	amount	amount
Control link	→ caller's frame	→ caller's frame	→ caller's frame
Access link	→ enclosing scope (if nested)	→ enclosing scope	→ enclosing scope
Saved registers	caller-saved regs	caller-saved regs	caller-saved regs
Local variables	internal computation temps	coupon lookup temps	invoice formatting temps
Return address	addr of instr after call	addr of instr after call	addr of instr after call

(iv) Sequence of Call/Return Operations — Stack Diagrams

```
Step 1: PUSH order              |  Step 2: CALL calculateBill
  [ caller frame        ]          [ caller frame        ]
  [ param: order        ]          [ AR: calculateBill   ]
                                   [   param: order      ]
                                   [   return addr        ]

Step 3: RETURN t1 (bill)        |  Step 4: amount = t1  (pop calculateBill AR)
  [ caller frame        ]          [ caller frame        ]
  [ t1 = returned value ]          [ amount = t1         ]

Step 5: PUSH amount, CALL applyCoupon      Step 6: RETURN (void), pop AR
  [ AR: applyCoupon      ]                  [ caller frame        ]
  [   param: amount      ]
  [   return addr         ]

Step 7: PUSH amount, CALL generateInvoice   Step 8: RETURN (void), pop AR
  [ AR: generateInvoice  ]                  [ caller frame        ]
  [   param: amount      ]
  [   return addr         ]
```

(v) Storage of Locals, Parameters and Return Address

- Actual parameters (order / amount) are stored at the base of the callee's activation record, at fixed offsets so the callee can access them via its frame pointer.
- Local variables used inside each function (intermediate temporaries for bill/coupon/invoice logic) are allocated above the parameters within the same activation record and are destroyed when the record is popped on return.
- The return address (the instruction to resume in the caller, e.g. the instruction following each call) is saved in the activation record when the call is made, and is used by the return instruction to transfer control back and pop the frame off the stack.
- Where a value is returned (calculateBill), a dedicated return-value slot in the activation record — or a designated register — carries the result back to the caller, where it is stored into t1 and then amount.

Q5. Smart Industrial Safety Monitoring System (10 Marks)

Concepts Covered: Assignment Statements, Boolean Expressions, Backpatching, Procedure Calls

(i) Boolean Expression

```
alarm_condition = (Temperature > 100) AND ( (Pressure > 80) OR (GasLeak == TRUE) )
```

(ii) Intermediate Code using Short-Circuit Evaluation

```
100: if Temperature > 100 goto 102
101: goto Lfalse                    // temperature check fails -> AND
short-circuits
102: if Pressure > 80 goto Ltrue     // OR: first disjunct true
```

```
103: if GasLeak == TRUE goto Ltrue    // OR: second disjunct
104: goto Lfalse                      // both OR operands false
```

(iii) Backpatching of Jump Targets

- E1 (Temperature>100).truelist = {100}, falselist = {101}
- E2 (Pressure>80 OR GasLeak==TRUE).truelist = {102, 103}, falselist = {104}
- E (overall AND).truelist = E2.truelist = {102, 103}
- E.falselist = merge(E1.falselist, E2.falselist) = {101, 104}

backpatch(E.truelist, 105) and backpatch(E.falselist, 108) are invoked once the target labels for the alarm-activation block (105) and the skip block (108) are known, filling those addresses into instructions 102, 103 (true) and 101, 104 (false).

(iv) Intermediate Code for Procedure Calls (condition true)

```
105: call activateAlarm, 0
106: call sendNotification, 0
107: goto 108
108: // next statement (condition false path also joins here)
```

(v) Complete Labelled Control Flow

```
100: if Temperature > 100 goto 102
101: goto 108
102: if Pressure > 80 goto 105
103: if GasLeak == TRUE goto 105
104: goto 108
105: call activateAlarm, 0
106: call sendNotification, 0
107: goto 108
108: // execution continues here in both cases
```

Execution sequence: the system first evaluates Temperature > 100 (100); if false it jumps straight to 108, skipping every other test (AND short-circuit). If true, it evaluates Pressure > 80 (102); if true it jumps directly to the alarm block at 105 without testing GasLeak (OR short-circuit). If Pressure > 80 is false, GasLeak == TRUE is checked (103); if true, control again reaches 105, otherwise it falls through to 108 with no alarm raised. When control reaches 105, activateAlarm() and sendNotification() are invoked in sequence via parameter-less procedure calls, after which control merges back to the common exit point at 108, ensuring the alarm fires only when the full safety condition is satisfied.